Viral Variant Atlas

Mutation Hotspot and Conservation Analysis Across Viral Gene Sequences

Technical Project Report

Adam Diri

Python | Bioinformatics | FASTA sequence analysis

September 2026

ILLUSTRATED EDITION

Four original figures, a reproducible synthetic demonstration, and complete Python source listings.

Biological observations from the original project remain preliminary. All new numerical charts are generated from explicitly synthetic inputs.

Abstract

Viral genomes accumulate substitutions as they replicate and circulate through host populations. Identifying where these substitutions recur is useful for exploratory surveillance because recurrent positions may indicate regions that tolerate variation, regions affected by selection, or positions requiring additional quality-control review. This project develops *Viral Variant Atlas*, a reproducible Python workflow for comparing viral gene sequences with a defined reference, reporting nucleotide substitutions with traceable coordinates, and ranking recurrent mutation positions across samples.

The original project concept used FASTA inputs, a sliding-window comparison, and CSV reporting to examine viral genes including ORF7b, ORF1ab, nucleocapsid phosphoprotein, surface glycoprotein, and other annotated loci. The upgraded workflow preserves that goal while making the analysis more reliable and transparent. It validates FASTA records, assumes homologous reference-query comparisons, evaluates both forward and reverse-complement orientations, applies configurable mismatch thresholds, records one-based reference positions, and separates individual mutation events from sample-level hotspot recurrence. The workflow produces three auditable artifacts: a mutation-level CSV, a ranked hotspot CSV, and a JSON run summary containing parameters and placement metadata.

The available project materials describe preliminary recurrent substitutions in ORF7b, including C51T and T54C, and also note comparatively conserved regions in nucleocapsid phosphoprotein, surface glycoprotein, and ORF1ab. Because the underlying FASTA files and raw result CSV were not supplied with this report, these observations are documented as prior project findings rather than independently recalculated results. The report therefore focuses on the analytical design, interpretation framework, quality controls, and limitations required to turn a one-off comparison script into a defensible exploratory bioinformatics project. The result is a portable, testable, and resume-ready sequence-analysis tool that supports transparent follow-up analysis while clearly distinguishing exploratory calls from biological or clinical conclusions.

Introduction and Background

Viral sequence variation and mutation hotspots

Sequence variation is central to viral evolution. During replication, errors introduced by polymerases, host-mediated editing, transmission bottlenecks, and selection can alter nucleotide sequences. Some changes are transient, whereas others recur in independent samples or persist in lineages. A recurrently observed nucleotide position is often described as a mutation hotspot, although recurrence alone does not prove an adaptive mechanism. A credible interpretation must consider sequencing quality, sample provenance, alignment quality, coding consequences, population structure, and the biological function of the affected locus [1].

For exploratory analysis, it is useful to distinguish two complementary patterns. First, recurrent substitutions identify positions that warrant follow-up because several samples carry a change at the same reference coordinate. Second, conserved positions identify regions that remain unchanged within the analyzed dataset. Conserved regions can be informative because strong functional constraints may limit tolerated variation; however, apparent conservation can also reflect small sample size, incomplete sampling, or a narrow comparison set. The purpose of this project is not to assign clinical significance to either pattern. Instead, it provides a reproducible way to describe the patterns accurately and generate well-structured files for downstream biological review.

Computational challenge

FASTA-style sequence files are simple to read but easy to analyze incorrectly. A script can silently compare a gene-length reference to an entire genome, report zero-based positions that appear one-based in a report, treat ambiguous bases as ordinary calls, or count multiple records from the same sample as independent recurrence. These issues affect reproducibility more than they affect code length. In particular, sliding-window matching is only meaningful when the reference and query are intended to represent homologous sequence regions. When insertions, deletions, structural changes, or distant homologs are expected, an ungapped comparison is insufficient and should be replaced by a validated alignment approach [2].

The original project correctly established several useful building blocks: FASTA loading, per-base mismatch identification, an allowed-mismatch rule, repeated comparison across reference and variant files, and CSV output. Viral Variant Atlas extends that foundation into a clearer analysis contract. The tool explicitly records the selected sequence orientation and reference offset, reports mutations in conventional REF-position-ALT notation, and retains run parameters with the output. These changes make the results easier to audit, reproduce, and explain during technical review.

Project objective and research questions

The objective is to develop a transparent command-line workflow that converts a defined set of viral reference and query sequences into mutation and hotspot reports. The project addresses four questions: (1) can FASTA inputs be validated before analysis; (2) can each query be placed reproducibly against its intended reference region; (3) can every nucleotide difference be recorded with unambiguous coordinates and provenance; and (4) can recurring positions be summarized without overstating biological significance?

The operational hypothesis is that an analysis pipeline with explicit input validation, orientation-aware placement, coordinate-correct reporting, and sample-level recurrence aggregation will be more reliable and interpretable than a single unstructured comparison loop. The biological hypothesis is intentionally modest: recurrent positions and conserved positions may identify candidates for further investigation, but no fitness, immune-escape, pathogenicity, or therapeutic claim is inferred from this workflow alone.

Methods

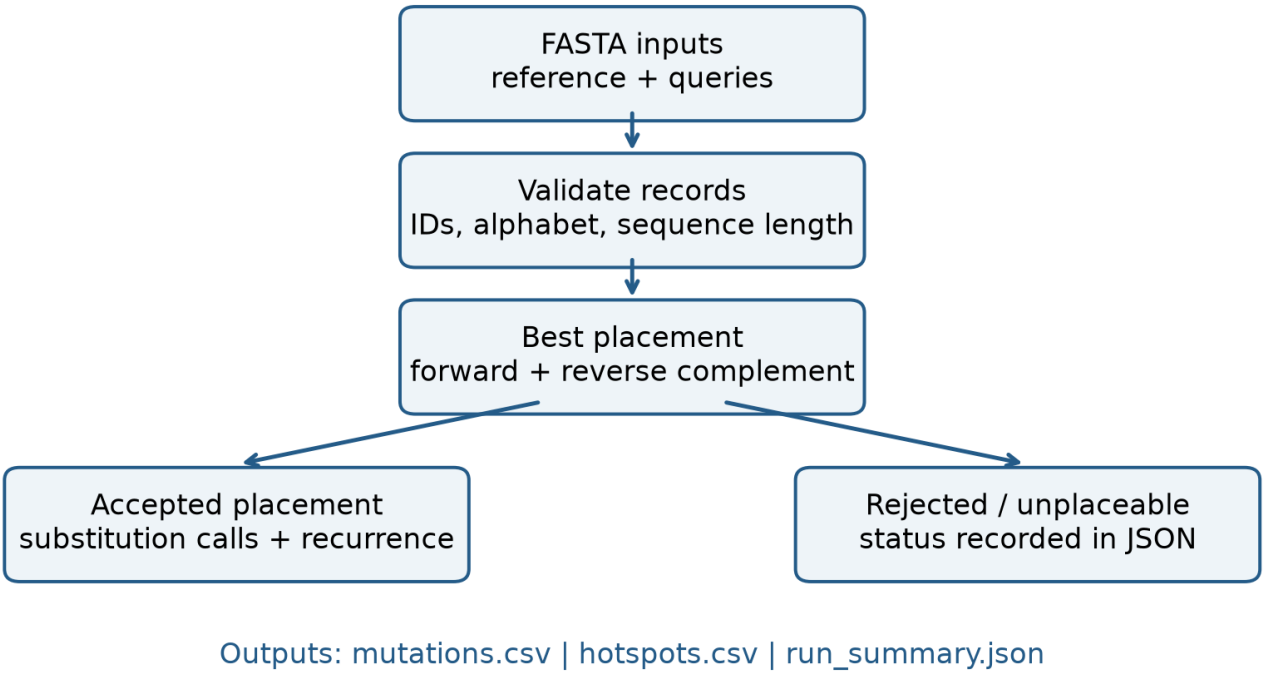


Figure 1. *Analysis workflow. Accepted placements feed the substitution and recurrence reports; rejected and unplaceable queries remain visible in the run summary.*

Dataset scope and organization

The project is designed for viral gene sequences stored in FASTA format. The source material lists reference genes such as ORF7b, ORF7a, ORF10, envelope protein, membrane glycoprotein, nucleocapsid phosphoprotein, ORF8, surface glycoprotein, ORF1a, ORF1ab, ORF3a, and ORF6. It also identifies five variant sequences by accession-style identifiers: PQ423980.1, OZ186360.1, OZ186552.1, PQ416396.1, and PQ422187.1. These identifiers are retained as project inputs, not independently verified biological annotations in this documentation pass.

Before analysis, each input must be assigned a clear role: reference or query; gene or whole genome; sample identifier; and, where available, collection metadata. A reference gene must be compared against the homologous locus in each query. Comparing a short reference gene against an unsegmented whole-genome FASTA can create misleading multiple placements and should be avoided unless the procedure deliberately locates that gene within each genome and records the coordinate system used. This report therefore treats locus matching as a required data-organization step rather than an implicit assumption.

Input component	Required role	Quality-control check
Reference FASTA	Single, named reference locus	Exactly one non-empty record with IUPAC DNA characters

Input component	Required role	Quality-control check
Query FASTA	One or more homologous sample loci	Stable sample IDs, valid alphabet, length recorded
Run parameters	Mismatch thresholds and reporting settings	Persisted in the JSON run summary
Results directory	Mutation and hotspot artifacts	Versioned or retained with input manifest

FASTA parsing and validation

The parser reads each header, stores a concise sequence identifier, joins sequence lines, removes whitespace, and normalizes bases to uppercase. It rejects files with no records, headers without identifiers, sequences before a header, empty sequences, and characters outside the IUPAC DNA alphabet. Validation is performed before any comparison begins so that an invalid file cannot generate an apparently successful but unreliable result. The current implementation permits IUPAC ambiguity symbols but compares them literally: an N-versus-A difference is a reported character mismatch, not a confirmed nucleotide substitution. The demonstration uses only A, C, G, and T. Ambiguity-aware masking remains future work.

Record identifiers are retained throughout the workflow. This is important because mutation calls without sample identity cannot be traced back to the source file. In the final CSV, every row includes a reference identifier, query identifier, orientation, reference position, reference base, alternate base, placement offset, and formatted substitution. The same identifiers are included in the run summary, allowing a reviewer to determine which input produced a particular call.

Orientation-aware ungapped placement

For each query, the program evaluates every window of equal length within the reference sequence. The Hamming distance between a query and each candidate reference window is calculated as the number of positions with different characters. The program repeats this evaluation using the reverse complement of the query. The best placement is the candidate with the smallest mismatch count. Ties are resolved deterministically: the lowest reference offset is selected first, and the forward orientation is preferred when mismatch count and offset are equal. Deterministic tie-breaking is essential because repeated analyses should not produce different outputs merely because candidate order changes.

If a query is longer than its reference, it is not silently truncated. Instead, the tool marks the record as unplaceable and records that decision in the run summary. This prevents a common error in which a partial comparison is treated as a complete match. The analysis accepts a placement only when the mismatch count is within a user-defined mismatch-rate threshold and, when supplied, an absolute mismatch cap. Unlike a blanket minimum of five allowed mismatches, a configurable threshold is easier to justify across loci of different lengths. For example, allowing five substitutions in a short gene may be a high mismatch rate, while the same absolute count may be negligible in a polyprotein sequence.

Mutation representation and coordinate convention

After selecting an accepted placement, the tool compares the aligned query window and reference bases position by position. Every character difference is stored as an event; calls involving ambiguity symbols require separate review. The formatted notation is REF-position-ALT, such as C51T, where C is the base in the reference, 51 is the one-based coordinate within the reference sequence, and T is the observed query base. The use of one-based coordinates matches common biological reporting practice and avoids the off-by-one ambiguity that arises when programming indexes are exposed directly.

Each event is also associated with the selected placement offset. This additional field allows a reviewer to determine whether a mutation position derives from an exact full-length comparison or a shorter query located within a larger reference. A no-mismatch result is represented by the absence of mutation rows for an accepted exact placement; it is not confused with a failed placement. Failed or over-threshold placements remain visible in the run summary so that missing mutation rows are not mistakenly interpreted as conservation.

Hotspot aggregation

A hotspot table is generated by grouping mutation events by reference identifier, reference position, and reference base. The primary recurrence measure is the number of unique query IDs carrying a change at that position. This sample-level count prevents duplicate rows from a single sample from inflating an apparent hotspot. The table also includes total event count and a compact distribution of substitution calls, for example T54C:2; T54G:1. Sorting is performed by decreasing sample recurrence, decreasing event count, and increasing reference position. The resulting order makes highly recurrent positions easy to inspect while remaining deterministic.

Recurrence is a descriptive statistic. It does not estimate mutation rate, selection coefficient, lineage frequency, or epidemiological prevalence. A recurrence count becomes biologically meaningful only when supported by representative sampling, validated sequence processing, appropriate alignment, and analysis of the corresponding coding or regulatory context. This distinction is carried throughout the report to prevent a computational summary from being mistaken for a clinical interpretation.

Outputs and reproducibility

Every successful run produces three files. **mutations.csv** contains one row per substitution. **hotspots.csv** contains the ranked recurrence summary. **run_summary.json** stores the reference ID and length, input query count, accepted-query count, number of mutation events, thresholds, and a placement record for every query. Together, these artifacts preserve both the detailed evidence and the high-level summary required for audit.

Output	Key fields	Use
mutations.csv	Sample ID, position, bases, orientation, offset	Trace individual calls to a source sequence
hotspots.csv	Position, unique-sample recurrence, substitution counts	Rank candidate recurrent regions

Output	Key fields	Use
run_summary.json	Parameters, placements, acceptance status, counts	Reproduce and audit a specific execution

The package is designed to run with the Python standard library and a small, explicit command-line interface. The absence of hidden notebook state reduces the risk that a report cannot be recreated outside the original development environment. Unit tests cover FASTA handling, reverse complements, placement offsets, coordinate formatting, threshold rejection, and unique-sample hotspot aggregation.

Synthetic demonstration and plotting procedure

A reproducible demonstration uses a 120-nucleotide artificial reference generated with Python random seed 772 and ten labeled query records. DEMO_01 is an exact reference match. DEMO_02 through DEMO_08 contain predefined substitutions at selected positions; DEMO_08 is supplied in reverse-complement orientation. DEMO_09 contains 30 introduced substitutions, and DEMO_10 is one nucleotide longer than the reference. These records are software fixtures, not patient samples, viral isolates, or accession-derived sequences.

The installed command-line package was run at a 5% mismatch threshold, permitting at most six mismatches in a 120-nucleotide comparison. The plotting script reads mutations.csv, hotspots.csv, and run_summary.json from that execution. Frequencies in Figure 2 use the eight accepted, full-length records as their denominator. Figure 3 shows the five positions with a substitution in at least one accepted record; it is not a whole-genome map. The input generator and every plotting command appear in Appendix C.

Results

Analytical outputs produced by the workflow

The upgraded workflow is structured to produce a complete evidence trail even when a sample does not contain an accepted placement. Accepted query records generate zero or more mutation rows; an accepted exact match therefore contributes to the conservation profile without requiring a misleading no-mismatch row. Rejected or unplaceable records remain listed in the JSON summary with mismatch count, offset, orientation, and acceptance status. For an unplaceable query, offset -1 is the status marker; its mismatch_count is a length sentinel, not a measured alignment distance. This separation makes the dataset easier to filter and protects against treating a comparison failure as a biologically conserved sequence.

The expected mutation-level output is concise and machine-readable. A representative row would identify the reference locus, the query sample, the selected orientation, the one-based reference position, the original and observed bases, the placement offset, and the formatted substitution. The expected hotspot-level output would summarize the same position across unique samples. These two levels support different questions: the first answers what changed in a specific sample; the second answers where changes recur in the observed set.

Previously reported project observations

The supplied project narrative reported recurrent substitutions around positions 51 to 54 in ORF7b, including C51T and T54C, across some variant comparisons. It also described comparatively conserved observations in nucleocapsid phosphoprotein, surface glycoprotein, and ORF1ab. These statements are retained as preliminary observations from the original project materials. They were not independently recalculated for this expanded report because the original FASTA inputs and generated gene_variant_comparison_results.csv file were not available in the workspace.

That limitation matters. Without the input sequences, one cannot verify the exact number of samples, whether each record corresponds to the same locus, which positions were compared after window placement, or whether any ambiguous bases were present. The report therefore supplies no new biological counts, statistical tests, mutation frequencies, or fitness claims. The separately labeled demonstration below uses artificial sequences to exercise the software. Instead, it provides the pipeline and reporting structure needed to reproduce the observation responsibly once the raw FASTA files and sample metadata are supplied.

Interpretation framework for hotspots and conservation

If repeated ORF7b substitutions are reproduced with validated homogeneous inputs, they would be reasonable candidates for follow-up. Follow-up could include translating the relevant coding region, determining whether the nucleotide change is synonymous or non-synonymous, mapping the amino-acid position, checking sequence coverage and quality scores, and examining whether the affected samples belong to one lineage or several. Recurrence across closely related samples may reflect shared ancestry rather than repeated independent mutation. Conversely, recurrence across multiple lineages might motivate a more detailed evolutionary analysis.

Similarly, an apparent absence of mutations in a region should be interpreted as conservation within the analyzed sample set, not universal conservation. The region may be conserved because it is functionally

constrained, because the comparison threshold excludes more divergent samples, or because the dataset lacks diversity. The output format makes these interpretations easier to test by preserving the included and excluded samples, rather than displaying only final mutation calls.

Reported pattern	Appropriate interpretation	Required next check
Repeated C51T or T54C in ORF7b	Candidate recurrent position in the project comparison set	Confirm with raw FASTA, quality metadata, and lineage context
No mutation row for an accepted exact match	Conserved relative to the selected reference under this run	Confirm placement and input coverage
Rejected query placement	Insufficient match for the configured ungapped comparison	Review locus identity or use a gapped aligner
High hotspot count	Descriptive recurrence, not proof of selection	Assess sampling and independent evidence

Synthetic demonstration: computed outputs

The actual demonstration run processed ten queries, accepted eight, rejected one above threshold, and marked one as unplaceable because it exceeded the reference length. The accepted records produced 13 substitution events across five reference positions. Position 12 recurred in five accepted samples; positions 35 and 61 each recurred in three; positions 88 and 109 each recurred in one. These counts describe the deliberately constructed software fixture only.

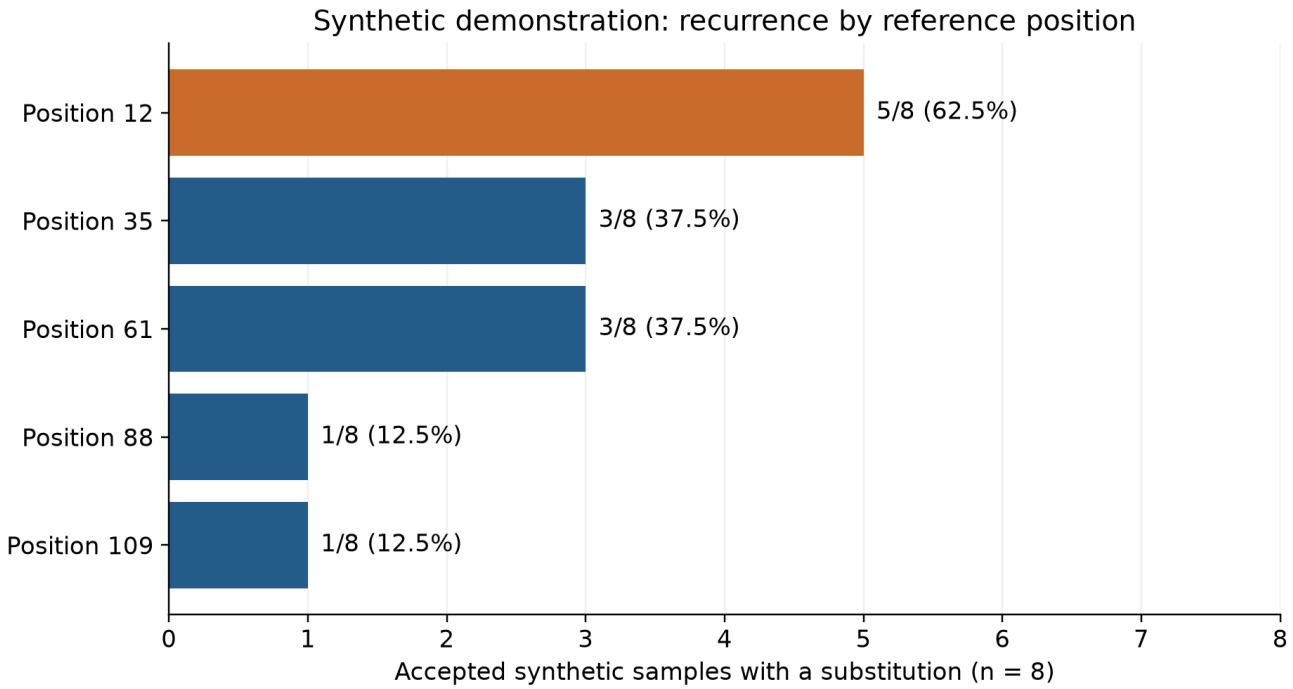


Figure 2. *Recurrence ranked by the number of unique accepted synthetic samples. The denominator is eight accepted full-length records; excluded records are omitted. Positions are coordinates in an artificial 120-nt reference, not ORF7b coordinates.*

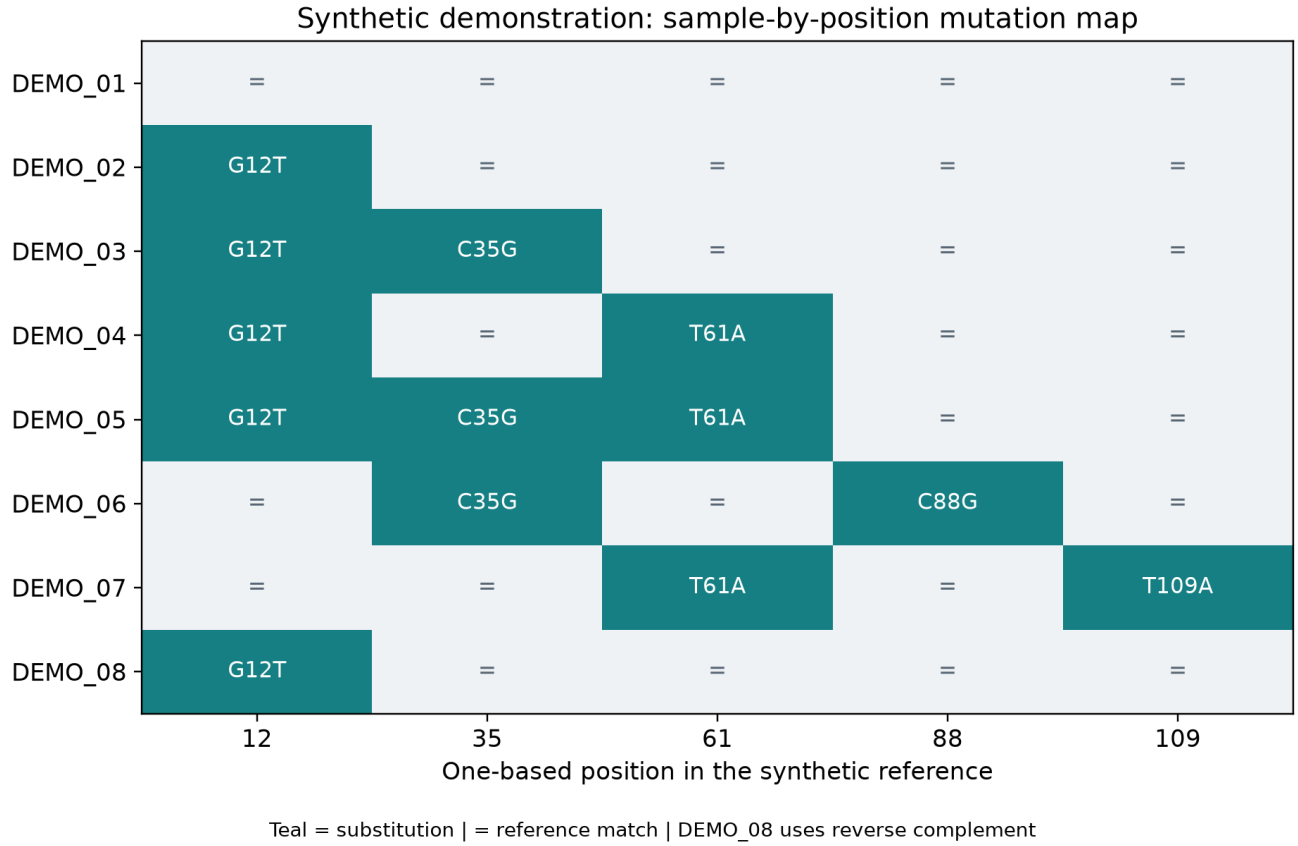


Figure 3. *Mutation map for the eight accepted synthetic samples and five variable positions. Teal cells show the actual reference-position-alternate call. An equals sign indicates a reference match at that shown position. DEMO_01 is an accepted exact match; DEMO_08 verifies reverse-complement handling.*

The matrix exposes the records responsible for each bar in Figure 2. It also retains the exact-match sample, demonstrating why a record with zero mutation rows must be interpreted together with its accepted placement status. The shared positions were introduced deliberately; their recurrence provides no evidence about viral evolution or selective pressure.

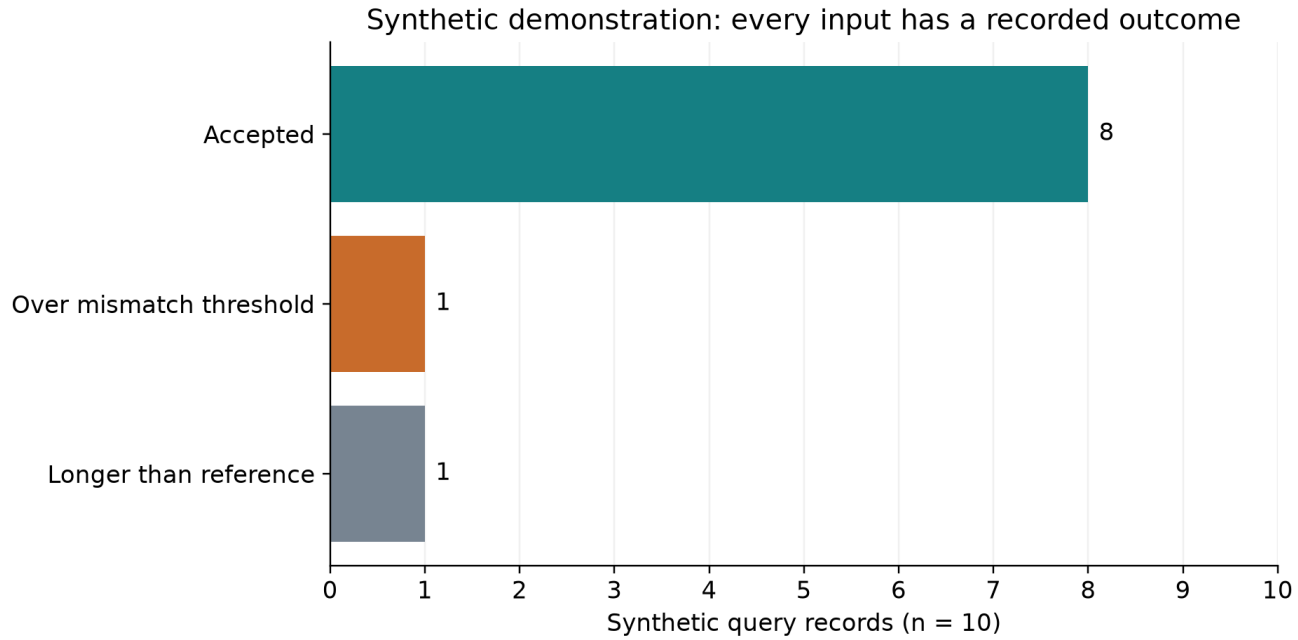


Figure 4. *Outcomes for all ten synthetic inputs at the 5% mismatch threshold. DEMO_09 fails the threshold, while DEMO_10 has no eligible reference window. Neither excluded sample contributes to hotspot recurrence.*

Execution and verification

The package executed successfully under Python 3.12.13. Ten unit tests passed, covering one-based coordinates, reverse-complement matching, deterministic ties, threshold rejection, the absolute mismatch cap, over-length queries, multi-record FASTA parsing, malformed inputs, and unique-sample aggregation. Separate end-to-end assertions verified the CLI's 10 total records, 8 accepted records, 13 mutation events, and recurrence counts of 5, 3, 3, 1, and 1. These are software checks on synthetic fixtures, not a biological accuracy estimate.

While preparing the executable appendix, the module entry point and empty-header validation were repaired, and the rejection fixture was corrected to account for reverse-complement matching. The source listings below correspond to the tested version used for the figures.

Discussion

Value of a reproducible exploratory workflow

The main contribution of Viral Variant Atlas is not a claim that a particular viral position is clinically important. Its contribution is methodological: it makes the assumptions behind a simple sequence-comparison task visible. The original code could identify per-character differences and save them to a CSV file, which is a practical first step. The upgraded design adds the details that make the output explainable: input identity, orientation, offset, a clearly defined coordinate system, explicit placement acceptance, and a sample-aware hotspot summary.

These improvements matter in educational and portfolio settings because they demonstrate more than scripting. They show how a computational method is framed, tested, and limited. A reviewer can inspect the output schema and understand what each row means. A future collaborator can rerun the command with the same inputs and parameters. A scientist can decide whether the reported calls are suitable for downstream alignment, annotation, or experimental validation. In this sense, reproducibility is itself a result of the project.

Methodological strengths

First, the parser fails early on malformed FASTA files. This avoids propagating hidden errors into a final report. Second, the search evaluates both sequence orientations. Although many curated viral-gene datasets are consistently oriented, an orientation check provides resilience when data are assembled from different sources. Third, deterministic tie-breaking means that a rerun cannot select a different equally scoring window without a documented reason. Fourth, the tool uses one-based reference coordinates and a conventional mutation notation, which makes the output easier to compare with biological notes and external annotations.

The hotspot aggregator also improves on a common reporting problem. Counting raw mutation rows can exaggerate recurrence when the same sample is duplicated, represented by multiple sequence records, or processed more than once. Counting unique query IDs is a more defensible descriptive measure. The raw event count is still retained so that a reviewer can identify discrepancies between sample recurrence and record-level volume.

Limitations

The primary limitation is the use of an ungapped sliding-window matcher. This approach is easy to understand and appropriate for closely related, equal-length regions with primarily substitution-level variation. It is not appropriate as a general replacement for pairwise or multiple sequence alignment. Insertions and deletions can shift the coordinate relationship between a reference and a query; a Hamming-distance window cannot represent those events. When indels are expected, a gapped alignment method such as a validated local or global aligner should be used [2].

A second limitation is biological context. The pipeline reports nucleotides, not amino-acid consequences. A nucleotide substitution in a coding region may be synonymous, non-synonymous, or affect an overlapping reading frame depending on annotation. The tool does not infer phenotype, transmissibility, immune escape, or treatment resistance. These interpretations require curated annotation, adequately sampled epidemiological data, and independent evidence.

A third limitation is dataset dependence. The available source description names several loci and preliminary observations, but it does not provide the raw sequences, alignment files, read-depth data, collection dates, or metadata. Therefore, this report does not claim a quantitative mutation rate, a significant hotspot, or a comparative frequency across variants. The report is deliberately explicit about that evidence boundary.

Future work

Several extensions would strengthen the project. The first is to integrate a gapped alignment engine and add explicit handling of insertions, deletions, and ambiguous positions. The second is to add coding-sequence annotation so that a nucleotide call can be paired with codon and amino-acid consequences when appropriate. The third is to add input manifests, checksums, software versions, and optional sample metadata so that results can be recreated across machines. The fourth is to apply the included visual summaries to validated biological datasets while keeping the underlying CSV as the source of truth.

For a larger research workflow, the project could also include lineage-aware summaries, position-level confidence flags, quality-filter integration, and statistical comparisons across defined groups. Those additions should be driven by the actual research question and available metadata rather than added solely to make the code appear more complex. The current design provides an appropriately scoped foundation for such work.

Conclusion

Viral Variant Atlas converts a basic FASTA comparison assignment into a reproducible exploratory bioinformatics workflow. It validates sequence inputs, evaluates forward and reverse-complement placements, records coordinate-correct nucleotide differences, summarizes recurring positions by unique sample, and preserves run settings with its outputs. The design supports transparent review of preliminary observations such as reported ORF7b substitutions while avoiding unsupported claims about biological significance.

The project also demonstrates a key principle of computational biology: a useful result is not only a list of differences. It is an evidence trail that states what was compared, how positions were defined, which records were accepted, what assumptions were made, and where the method should not be over-interpreted. With raw FASTA files and sample metadata, this workflow can be rerun to generate a traceable mutation and conservation report suitable for downstream alignment, annotation, and scientific review.

References

[1] Rogozin, I. B., and Pavlov, Y. I. (2003). Theoretical analysis of mutation hotspots and their DNA sequence context specificity. *Mutation Research*, 544(1), 65-85. https://doi.org/10.1016/S1383-5742(03)00032-2

[2] Altschul, S. F., Gish, W., Miller, W., Myers, E. W., and Lipman, D. J. (1990). Basic local alignment search tool. *Journal of Molecular Biology*, 215(3), 403-410. https://doi.org/10.1016/S0022-2836(05)80360-2

[3] Cock, P. J. A., et al. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. https://doi.org/10.1093/bioinformatics/btp163

[4] O'Leary, N. A., et al. (2016). Reference sequence (RefSeq) database at NCBI: current status, taxonomic expansion, and functional annotation. *Nucleic Acids Research*, 44(D1), D733-D745. https://doi.org/10.1093/nar/gkv1189

Appendix A: Reproducible command-line workflow

Extract the accompanying source bundle and open a terminal in the viral-variant-atlas directory. The core package uses the Python standard library; matplotlib is required only for figure generation. Commands below work as separate lines in PowerShell or a conventional terminal. Python 3.10 or newer is required.

```
python -m pip install -e .
python -m pip install matplotlib==3.11.2
python -m unittest discover -s tests -v
python scripts/make_demo_figures.py
```

The last command regenerates the synthetic FASTA inputs, executes the CLI, checks expected counts, and writes all four PNG figures. Use the following command, on one line, for the small original example supplied in the repository:

```
python -m viral_variant_atlas --reference data/example/reference_orf7b.fasta
  --queries data/example/variant_a.fasta data/example/variant_b.fasta
  --output-dir results/example --max-mismatch-rate 0.10
```

The command is wrapped visually above; join its three lines with spaces. To analyze real data, replace the reference and query paths with validated homologous loci. A threshold is an analytical parameter and must be chosen for the study. The synthetic reference is not intended as a biological reference.

The source bundle includes the package, tests, figure-generation script, synthetic FASTA inputs, generated CSV/JSON outputs, and PNG images. The core package can run without plotting libraries. The complete listings are provided in Appendix C so the methods and figures can be reviewed alongside the code.

Appendix B: Output schema

Field	Description
reference_id	Identifier of the reference FASTA record
query_id	Identifier of the analyzed sample record
orientation	Forward (+) or reverse-complement (-) placement
reference_position	One-based position within the reference record
reference_base / alternate_base	Nucleotide call at the selected position
placement_offset	Zero-based start offset of the selected query window
substitution	Human-readable call, for example C51T
sample_recurrence	Number of unique query IDs carrying a change at a position

Data-availability note: This report documents the project methodology and the prior observations described in the supplied project materials. Raw FASTA sequences, the original generated CSV, sample metadata, and quality metrics were not available for independent reanalysis in this report.

Appendix C: Complete source code

The listings are copied directly from the tested source files. Line numbers are for reading only and are not part of the code. Use the .py files in the accompanying source bundle for execution. The listing includes the full analysis package, command entry point, automated tests, configuration, and synthetic-data/figure generator.

C.1 Package and configuration

Listing C.1. pyproject.toml

```
 1  [build-system]
 2  requires = ["setuptools>=68"]
 3  build-backend = "setuptools.build_meta"
 4
 5  [project]
 6  name = "viral-variant-atlas"
 7  version = "1.0.0"
 8  description = "Reproducible FASTA comparison and mutation-hotspot reporting"
 9  requires-python = ">=3.10"
10
11  [project.scripts]
12  viral-variant-atlas = "viral_variant_atlas.cli:main"
13
14  [tool.setuptools]
15  package-dir = {"" = "src"}
16
17  [tool.setuptools.packages.find]
18  where = ["src"]
```

Listing C.2. src/viral_variant_atlas/__init__.py

```
"""Viral Variant Atlas: traceable, ungapped FASTA sequence comparison."""

__version__ = "1.0.0"
```

Listing C.3. src/viral_variant_atlas/__main__.py

```
from .cli import main

if __name__ == "__main__":
    main()
```

Listing C.4. src/viral_variant_atlas/types.py

```
from __future__ import annotations

from dataclasses import asdict, dataclass


@dataclass(frozen=True)
class FastaRecord:
    identifier: str
    sequence: str


@dataclass(frozen=True)
class Placement:
    query_id: str
    orientation: str
    offset: int
    mismatch_count: int
    accepted: bool


@dataclass(frozen=True)
class Mutation:
    reference_id: str
    query_id: str
    orientation: str
    reference_position: int
    reference_base: str
    alternate_base: str
    placement_offset: int

    @property
    def substitution(self) -> str:
        return f"{self.reference_base}{self.reference_position}{self.alternate_base}"

    def row(self) -> dict[str, object]:
        row = asdict(self)
        row["substitution"] = self.substitution
        return row
```

Listing C.5. src/viral_variant_atlas/fasta.py

```
from __future__ import annotations

from pathlib import Path

from .types import FastaRecord

IUPAC_DNA = frozenset("ACGTRYSWKMBDHVN")


class FastaFormatError(ValueError):
    """Raised when FASTA content is malformed or unsupported."""


def read_fasta(path: str | Path) -> list[FastaRecord]:
    """Parse a FASTA file, retaining record IDs and validating DNA characters."""
    source = Path(path)
    records: list[FastaRecord] = []
    identifier: str | None = None
    fragments: list[str] = []

    def flush() -> None:
        nonlocal fragments
        if identifier is None:
            return
        sequence = "".join(fragments).upper()
        if not sequence:
            raise FastaFormatError(
                f"{source}: record '{identifier}' has no sequence"
            )
        invalid = sorted(set(sequence) - IUPAC_DNA)
        if invalid:
            raise FastaFormatError(
                f"{source}: record '{identifier}' has unsupported bases: {''.join(invalid)}"
            )
        records.append(
            FastaRecord(identifier=identifier, sequence=sequence)
        )
        fragments = []

    for line_number, raw_line in enumerate(
        source.read_text(encoding="utf-8").splitlines(), 1
    ):
        line = raw_line.strip()
        if not line:
            continue
        if line.startswith(">"):
            flush()
            header_parts = line[1:].split()
            if not header_parts:
                raise FastaFormatError(
                    f"{source}:{line_number}: FASTA header has no identifier"
                )
            identifier = header_parts[0]
        elif identifier is None:
            raise FastaFormatError(
                f"{source}:{line_number}: sequence appears before a FASTA header"
            )
        else:
            fragments.append("".join(line.split()))
    flush()
    if not records:
        raise FastaFormatError(f"{source}: no FASTA records found")
    return records
```

Listing C.6. src/viral_variant_atlas/analysis.py

```
from __future__ import annotations

from collections import Counter, defaultdict
from dataclasses import asdict

from .types import FastaRecord, Mutation, Placement

_COMPLEMENT = str.maketrans("ACGTRYSWKMBDHVN", "TGCAYRSWMKVHDBN")


def reverse_complement(sequence: str) -> str:
    return sequence.translate(_COMPLEMENT)[::-1]


def allowed_mismatches(
    query_length: int, rate: float, absolute_cap: int | None
) -> int:
    if not 0 <= rate <= 1:
        raise ValueError("max mismatch rate must be between 0 and 1")
    if absolute_cap is not None and absolute_cap < 0:
        raise ValueError("absolute mismatch cap must be non-negative")
    threshold = int(query_length * rate)
    return (
        min(threshold, absolute_cap)
        if absolute_cap is not None
        else threshold
    )


def best_placement(
    reference: str,
    query: FastaRecord,
    rate: float,
    absolute_cap: int | None,
) -> Placement:
    """Find the minimum-Hamming-distance placement across both strands."""
    threshold = allowed_mismatches(len(query.sequence), rate, absolute_cap)
    if len(query.sequence) > len(reference):
        return Placement(
            query.identifier, "+", -1, len(query.sequence), False
        )
    candidates: list[tuple[int, int, str]] = []
    for orientation, sequence in (
        ("+", query.sequence),
        ("-", reverse_complement(query.sequence)),
    ):
        for offset in range(len(reference) - len(sequence) + 1):
            mismatches = sum(
                a != b
                for a, b in zip(
                    reference[offset : offset + len(sequence)], sequence
                )
            )
            candidates.append((mismatches, offset, orientation))
    mismatch_count, offset, orientation = min(
        candidates, key=lambda item: (item[0], item[1], item[2] != "+")
    )
    return Placement(
        query.identifier,
        orientation,
        offset,
        mismatch_count,
        mismatch_count <= threshold,
```

```
    )


def call_mutations(
    reference: FastaRecord, query: FastaRecord, placement: Placement
) -> list[Mutation]:
    if not placement.accepted:
        return []
    sequence = (
        query.sequence
        if placement.orientation == "+"
        else reverse_complement(query.sequence)
    )
    window = reference.sequence[
        placement.offset : placement.offset + len(sequence)
    ]
    return [
        Mutation(
            reference.identifier,
            query.identifier,
            placement.orientation,
            placement.offset + index + 1,
            ref_base,
            alt_base,
            placement.offset,
        )
        for index, (ref_base, alt_base) in enumerate(zip(window, sequence))
        if ref_base != alt_base
    ]


def hotspot_rows(mutations: list[Mutation]) -> list[dict[str, object]]:
    grouped: dict[tuple[str, int, str], list[Mutation]] = defaultdict(list)
    for mutation in mutations:
        grouped[
            (
                mutation.reference_id,
                mutation.reference_position,
                mutation.reference_base,
            )
        ].append(mutation)
    rows: list[dict[str, object]] = []
    for (reference_id, position, base), events in grouped.items():
        substitutions = Counter(event.substitution for event in events)
        rows.append(
            {
                "reference_id": reference_id,
                "reference_position": position,
                "reference_base": base,
                "sample_recurrence": len(
                    {event.query_id for event in events}
                ),
                "event_count": len(events),
                "substitution_counts": "; ".join(
                    f"{call}:{count}"
                    for call, count in sorted(substitutions.items())
                ),
            }
        )
    return sorted(
        rows,
        key=lambda row: (
            -int(row["sample_recurrence"]),
            -int(row["event_count"]),
            int(row["reference_position"]),
```

```
        ),
    )


def placement_row(placement: Placement) -> dict[str, object]:
    return asdict(placement)
```

Listing C.7. src/viral_variant_atlas/cli.py

```
from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path

from .analysis import (
    best_placement,
    call_mutations,
    hotspot_rows,
    placement_row,
)
from .fasta import read_fasta


def _write_csv(
    path: Path, rows: list[dict[str, object]], fields: list[str]
) -> None:
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Compare FASTA queries and rank recurrent mutation hotspots."
    )
    parser.add_argument(
        "--reference",
        required=True,
        help="Reference FASTA containing exactly one record.",
    )
    parser.add_argument(
        "--queries",
        required=True,
        nargs="+",
        help="One or more query FASTA files.",
    )
    parser.add_argument(
        "--output-dir",
        required=True,
        help="Directory for CSV and JSON results.",
    )
    parser.add_argument(
        "--max-mismatch-rate",
        type=float,
        default=0.01,
        help="Maximum allowed mismatch rate (default: 0.01).",
    )
    parser.add_argument(
        "--absolute-mismatch-cap",
        type=int,
        default=None,
        help="Optional maximum mismatch count.",
    )
    parser.add_argument(
        "--top-hotspots",
        type=int,
        default=20,
        help="Hotspots retained in run summary (default: 20).",
    )
```

```
    args = parser.parse_args()
    if not 0 <= args.max_mismatch_rate <= 1:
        parser.error("--max-mismatch-rate must be between 0 and 1")
    if (
        args.absolute_mismatch_cap is not None
        and args.absolute_mismatch_cap < 0
    ):
        parser.error("--absolute-mismatch-cap must be non-negative")
    if args.top_hotspots < 0:
        parser.error("--top-hotspots must be non-negative")
    return args


def main() -> None:
    args = parse_args()
    references = read_fasta(args.reference)
    if len(references) != 1:
        raise ValueError("Reference FASTA must contain exactly one record")
    reference = references[0]
    queries = [
        record
        for file_name in args.queries
        for record in read_fasta(file_name)
    ]
    if len({record.identifier for record in queries}) != len(queries):
        raise ValueError(
            "Query IDs must be unique; remove duplicate samples first"
        )
    placements = [
        best_placement(
            reference.sequence,
            query,
            args.max_mismatch_rate,
            args.absolute_mismatch_cap,
        )
        for query in queries
    ]
    mutations = [
        mutation
        for query, placement in zip(queries, placements)
        for mutation in call_mutations(reference, query, placement)
    ]
    hotspots = hotspot_rows(mutations)
    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    _write_csv(
        output_dir / "mutations.csv",
        [item.row() for item in mutations],
        [
            "reference_id",
            "query_id",
            "orientation",
            "reference_position",
            "reference_base",
            "alternate_base",
            "placement_offset",
            "substitution",
        ],
    )
    _write_csv(
        output_dir / "hotspots.csv",
        hotspots,
        [
            "reference_id",
            "reference_position",
```

```
            "reference_base",
            "sample_recurrence",
            "event_count",
            "substitution_counts",
        ],
    )
    summary = {
        "reference_id": reference.identifier,
        "reference_length": len(reference.sequence),
        "query_count": len(queries),
        "accepted_query_count": sum(p.accepted for p in placements),
        "mutation_count": len(mutations),
        "parameters": {
            "max_mismatch_rate": args.max_mismatch_rate,
            "absolute_mismatch_cap": args.absolute_mismatch_cap,
        },
        "placements": [placement_row(item) for item in placements],
        "top_hotspots": hotspots[: args.top_hotspots],
    }
    (output_dir / "run_summary.json").write_text(
        json.dumps(summary, indent=2) + "\n", encoding="utf-8"
    )
    print(
        f"Processed {len(queries)} query records; wrote results to {output_dir}"
    )


if __name__ == "__main__":
    main()
```

Listing C.8. tests/test_analysis.py

```
import unittest

from viral_variant_atlas.analysis import (
    best_placement,
    call_mutations,
    hotspot_rows,
    reverse_complement,
)
from viral_variant_atlas.types import FastaRecord


class AnalysisTests(unittest.TestCase):
    def test_calls_one_based_substitution_at_best_offset(self):
        reference = FastaRecord("ref", "GGATGCTACCTT")
        query = FastaRecord("sample", "ATGCTTC")
        placement = best_placement(
            reference.sequence, query, rate=0.20, absolute_cap=None
        )
        self.assertEqual(
            (
                placement.offset,
                placement.orientation,
                placement.mismatch_count,
            ),
            (2, "+", 1),
        )
        self.assertEqual(
            [
                event.substitution
                for event in call_mutations(reference, query, placement)
            ],
            ["A8T"],
        )

    def test_considers_reverse_complement(self):
        self.assertEqual(reverse_complement("ATGC"), "GCAT")
        placement = best_placement(
            "CCGCATGG",
            FastaRecord("sample", "ATGC"),
            rate=0,
            absolute_cap=None,
        )
        self.assertEqual(
            (
                placement.orientation,
                placement.offset,
                placement.mismatch_count,
            ),
            ("-", 2, 0),
        )

    def test_hotspots_count_unique_samples(self):
        reference = FastaRecord("ref", "ATGC")
        queries = [FastaRecord("a", "ATAC"), FastaRecord("b", "ATAC")]
        mutations = [
            event
            for query in queries
            for event in call_mutations(
                reference,
                query,
                best_placement(reference.sequence, query, 0.5, None),
            )
        ]
```

```
        hotspot = hotspot_rows(mutations)[0]
        self.assertEqual(
            (
                hotspot["reference_position"],
                hotspot["sample_recurrence"],
                hotspot["substitution_counts"],
            ),
            (3, 2, "G3A:2"),
        )

    def test_rejects_over_threshold_placement(self):
        placement = best_placement(
            "AAAA",
            FastaRecord("sample", "CCCC"),
            rate=0.25,
            absolute_cap=None,
        )
        self.assertFalse(placement.accepted)

    def test_reverse_complement_exact_match_is_accepted(self):
        result = best_placement(
            "AAAA", FastaRecord("sample", "TTTT"), 0, None
        )
        self.assertTrue(result.accepted)
        self.assertEqual(result.orientation, "-")

    def test_tie_prefers_lowest_offset_and_forward(self):
        result = best_placement(
            "ATATAT", FastaRecord("sample", "ATAT"), 0, None
        )
        self.assertEqual((result.offset, result.orientation), (0, "+"))

    def test_long_query_is_unplaceable(self):
        result = best_placement(
            "ATGC", FastaRecord("sample", "ATGCA"), 0, None
        )
        self.assertEqual((result.offset, result.accepted), (-1, False))

    def test_absolute_cap_rejects_otherwise_accepted_query(self):
        result = best_placement("AAAA", FastaRecord("sample", "AACC"), 1, 1)
        self.assertFalse(result.accepted)


if __name__ == "__main__":
    unittest.main()
```

Listing C.9. tests/test_fasta.py

```
import tempfile
import unittest
from pathlib import Path

from viral_variant_atlas.fasta import FastaFormatError, read_fasta


class FastaTests(unittest.TestCase):
    def parse(self, text):
        with tempfile.TemporaryDirectory() as directory:
            path = Path(directory) / "input.fasta"
            path.write_text(text, encoding="utf-8")
            return read_fasta(path)

    def test_multiple_records_and_whitespace(self):
        records = self.parse(">a description\nat g\tc\n>b\nNNRY\n")
        self.assertEqual(
            [(r.identifier, r.sequence) for r in records],
            [("a", "ATGC"), ("b", "NNRY")],
        )

    def test_malformed_inputs_are_rejected(self):
        for text in ("", ">\nACGT", ">a\n", "ACGT", ">a\nACGT!"):
            with self.subTest(text=text), self.assertRaises(
                FastaFormatError
            ):
                self.parse(text)
```

Listing C.10. scripts/make_demo_figures.py

```
"""Generate synthetic FASTA data, run the CLI, and plot its actual outputs.

Run from the repository root after installing the package and matplotlib.
Every sequence below is artificial and has no viral accession or annotation.
"""

import csv
import json
import random
import subprocess
import sys
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from matplotlib.patches import FancyBboxPatch

from viral_variant_atlas.analysis import reverse_complement

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data" / "synthetic"
RESULTS = ROOT / "results" / "synthetic"
FIGURES = ROOT / "figures"
BLUE, TEAL, ORANGE = "#245B88", "#157F83", "#C86B2B"
plt.rcParams.update({"font.size": 11, "font.family": "DejaVu Sans"})


def save(fig, name):
    fig.savefig(FIGURES / name, dpi=220, bbox_inches="tight")
    plt.close(fig)


def make_data():
    rng = random.Random(772)
    reference = "".join(rng.choice("ACGT") for _ in range(120))
    changes = [
        [],
        [12],
        [12, 35],
        [12, 61],
        [12, 35, 61],
        [35, 88],
        [61, 109],
        [12],
    ]
    alternate = dict(zip("ACGT", "CGTA"))
    records = []
    for number, positions in enumerate(changes, 1):
        bases = list(reference)
        for position in positions:
            bases[position - 1] = alternate[bases[position - 1]]
        sequence = "".join(bases)
        if number == 8:
            sequence = reverse_complement(sequence)
        records.append((f"DEMO_{number:02}", sequence))
    divergent = list(reference)
    for index in range(0, 120, 4):
        divergent[index] = alternate[divergent[index]]
    records += [
        ("DEMO_09", "".join(divergent)),
```

```
        ("DEMO_10", reference + "A"),
    ]
    (DATA / "reference.fasta").write_text(
        ">SYNTHETIC_REFERENCE_120NT\n" + reference + "\n",
        encoding="utf-8",
    )
    (DATA / "queries.fasta").write_text(
        "".join(f">{name}\n{seq}\n" for name, seq in records),
        encoding="utf-8",
    )


def main():
    for directory in (DATA, RESULTS, FIGURES):
        directory.mkdir(parents=True, exist_ok=True)
    make_data()
    command = [
        sys.executable,
        "-m",
        "viral_variant_atlas",
        "--reference",
        str(DATA / "reference.fasta"),
        "--queries",
        str(DATA / "queries.fasta"),
        "--output-dir",
        str(RESULTS),
        "--max-mismatch-rate",
        "0.05",
    ]
    subprocess.run(command, check=True)
    summary = json.loads((RESULTS / "run_summary.json").read_text())
    with (RESULTS / "mutations.csv").open(newline="") as handle:
        mutations = list(csv.DictReader(handle))
    with (RESULTS / "hotspots.csv").open(newline="") as handle:
        hotspots = list(csv.DictReader(handle))
    assert summary["query_count"] == 10
    assert summary["accepted_query_count"] == 8
    assert summary["mutation_count"] == 13
    assert [
        (int(h["reference_position"]), int(h["sample_recurrence"]))
        for h in hotspots
    ] == [(12, 5), (35, 3), (61, 3), (88, 1), (109, 1)]

    # Figure 1: explicit branching avoids equating rejection with conservation.
    fig, ax = plt.subplots(figsize=(9, 5.2))
    ax.set(xlim=(0, 10), ylim=(0, 7))
    ax.axis("off")
    boxes = [
        (3.3, 5.8, "FASTA inputs\nreference + queries"),
        (3.3, 4.4, "Validate records\nIDs, alphabet, sequence length"),
        (3.3, 3.0, "Best placement\nforward + reverse complement"),
        (0.2, 1.4, "Accepted placement\nsubstitution calls + recurrence"),
        (6.4, 1.4, "Rejected / unplaceable\nstatus recorded in JSON"),
    ]
    for x, y, label in boxes:
        patch = FancyBboxPatch(
            (x, y),
            3.4,
            0.85,
            boxstyle="round,pad=0.12",
            facecolor="#EEF4F8",
            edgecolor=BLUE,
            linewidth=1.3,
        )
        ax.add_patch(patch)
```

```
        ax.text(x + 1.7, y + 0.43, label, ha="center", va="center")
    for start, end in [
        ((5, 5.8), (5, 5.35)),
        ((5, 4.4), (5, 3.95)),
        ((4.3, 3), (1.9, 2.4)),
        ((5.7, 3), (8.1, 2.4)),
    ]:
        ax.annotate(
            "",
            xy=end,
            xytext=start,
            arrowprops=dict(arrowstyle="->", color=BLUE, lw=1.6),
        )
    ax.text(
        5,
        0.5,
        "Outputs: mutations.csv | hotspots.csv | run_summary.json",
        ha="center",
        color=BLUE,
    )
    save(fig, "01_pipeline.png")

    # Figure 2: rank positions using counts read from hotspots.csv.
    fig, ax = plt.subplots(figsize=(8.6, 4.7))
    values = [int(row["sample_recurrence"]) for row in hotspots]
    labels = [f"Position {row['reference_position']}" for row in hotspots]
    bars = ax.barh(labels, values, color=[ORANGE] + [BLUE] * 4)
    ax.invert_yaxis()
    ax.bar_label(
        bars, labels=[f"{v}/8 ({v / 8:.1%})" for v in values], padding=6
    )
    ax.set(
        xlim=(0, 8),
        xticks=range(9),
        xlabel="Accepted synthetic samples with a substitution (n = 8)",
        title="Synthetic demonstration: recurrence by reference position",
    )
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(axis="x", alpha=0.18)
    ax.set_axisbelow(True)
    fig.tight_layout()
    save(fig, "02_recurrence.png")

    # Figure 3: show accepted exact matches separately from rejected records.
    samples = [f"DEMO_{i:02}" for i in range(1, 9)]
    positions = [12, 35, 61, 88, 109]
    calls = {
        (r["query_id"], int(r["reference_position"])): r["substitution"]
        for r in mutations
    }
    matrix = [[int((s, p) in calls) for p in positions] for s in samples]
    fig, ax = plt.subplots(figsize=(8.6, 5.6))
    ax.imshow(
        matrix,
        cmap=ListedColormap(["#EEF2F5", TEAL]),
        vmin=0,
        vmax=1,
        aspect="auto",
    )
    for row, sample in enumerate(samples):
        for col, position in enumerate(positions):
            changed = (sample, position) in calls
            ax.text(
                col,
                row,
```

```
                calls.get((sample, position), "="),
                ha="center",
                va="center",
                fontsize=10,
                color="white" if changed else "#586674",
            )
    ax.set(
        xticks=range(5),
        xticklabels=positions,
        yticks=range(8),
        yticklabels=samples,
        xlabel="One-based position in the synthetic reference",
        title="Synthetic demonstration: sample-by-position mutation map",
    )
    fig.text(
        0.5,
        0.015,
        "Teal = substitution | = reference match | DEMO_08 uses reverse complement",
        ha="center",
        fontsize=9,
    )
    fig.tight_layout(rect=(0, 0.045, 1, 1))
    save(fig, "03_mutation_map.png")

    # Figure 4: report all inputs, including those with no accepted placement.
    placements = summary["placements"]
    values = [
        sum(p["accepted"] for p in placements),
        sum(not p["accepted"] and p["offset"] >= 0 for p in placements),
        sum(p["offset"] < 0 for p in placements),
    ]
    fig, ax = plt.subplots(figsize=(8.6, 4.4))
    bars = ax.barh(
        ["Accepted", "Over mismatch threshold", "Longer than reference"],
        values,
        color=[TEAL, ORANGE, "#778491"],
    )
    ax.invert_yaxis()
    ax.bar_label(bars, padding=5)
    ax.set(
        xlim=(0, 10),
        xticks=range(11),
        xlabel="Synthetic query records (n = 10)",
        title="Synthetic demonstration: every input has a recorded outcome",
    )
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(axis="x", alpha=0.18)
    ax.set_axisbelow(True)
    fig.tight_layout()
    save(fig, "04_query_outcomes.png")
    print(
        json.dumps(
            {
                "accepted": values[0],
                "events": len(mutations),
                "hotspots": len(hotspots),
                "figures": 4,
            }
        )
    )


if __name__ == "__main__":
    main()
```